



3.10.13



Q Recherche rapide

Go

## 6. Modules

Lorsque vous quittez et entrez à nouveau dans l'interpréteur Python, tout ce que vous avez déclaré dans la session précédente est perdu. Afin de rédiger des programmes plus longs, vous devez utiliser un éditeur de texte, préparer votre code dans un fichier et exécuter Python avec ce fichier en paramètre. Cela s'appelle créer un *script*. Lorsque votre programme grandit, vous pouvez séparer votre code dans plusieurs fichiers. Ainsi, il vous est facile de réutiliser des fonctions écrites pour un programme dans un autre sans avoir à les copier.

Pour gérer cela, Python vous permet de placer des définitions dans un fichier et de les utiliser dans un script ou une session interactive. Un tel fichier est appelé un *module* et les définitions d'un module peuvent être importées dans un autre module ou dans le module *main* (qui est le module qui contient vos variables et définitions lors de l'exécution d'un script au niveau le plus haut ou en mode interactif).

Un module est un fichier contenant des définitions et des instructions. Son nom de fichier est le nom du module suffixé de `.py`. À l'intérieur d'un module, son propre nom est accessible par la variable `__name__`. Par exemple, prenez votre éditeur favori et créez un fichier `fibonacci.py` dans le répertoire courant qui contient :

```
# Fibonacci numbers module

def fib(n):    # write Fibonacci series up to n
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()

def fib2(n):   # return Fibonacci series up to n
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a+b
    return result
```

Maintenant, ouvrez un interpréteur et importez le module en tapant :

```
>>> import fibo
```

&gt;&gt;&gt;

Les noms des fonctions définies dans `fibo` ne sont pas ajoutés directement dans l'[espace de nommage](#) courant (voir [Portées et espaces de nommage en Python](#) pour plus de détails), seul le nom de module `fibo` est ajouté. L'appel des fonctions se fait donc *via* le nom du module :

```
>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

&gt;&gt;&gt;

Si vous avez l'intention d'utiliser souvent une fonction, il est possible de lui assigner un nom local :

```
>>> fib = fibo.fib
>>> fib(500)
```

&gt;&gt;&gt;



## 6.1. Les modules en détail

Un module peut contenir aussi bien des instructions que des déclarations de fonctions. Ces instructions permettent d'initialiser le module. Elles ne sont exécutées que la *première* fois que le nom d'un module est trouvé dans un `import` [1] (elles sont aussi exécutées lorsque le fichier est exécuté en tant que script).

Chaque module possède son propre espace de nommage, utilisé comme espace de nommage global par toutes les fonctions définies par le module. Ainsi l'auteur d'un module peut utiliser des variables globales dans un module sans se soucier de collisions de noms avec des variables globales définies par l'utilisateur du module. Cependant, si vous savez ce que vous faites, vous pouvez modifier une variable globale d'un module avec la même notation que pour accéder aux fonctions : `nommodule.nomelement`.

Des modules peuvent importer d'autres modules. Il est courant, mais pas obligatoire, de ranger tous les `import` au début du module (ou du script). Les noms des modules importés, s'ils sont placés au début d'un module (en dehors des fonctions et des classes) sont ajoutés à l'espace de nommage global du module.

Il existe une variante de l'instruction `import` qui importe les noms d'un module directement dans l'espace de nommage du module qui l'importe, par exemple :

```
>>> from fibo import fib, fib2
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

&gt;&gt;&gt;

Cela n'insère pas le nom du module depuis lequel les définitions sont récupérées dans l'espace de nommage local (dans cet exemple, `fibo` n'est pas défini).

Il existe même une variante permettant d'importer tous les noms qu'un module définit :

```
>>> from fibo import *
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

&gt;&gt;&gt;

Tous les noms ne commençant pas par un tiret bas (`_`) sont importés. Dans la grande majorité des cas, les développeurs n'utilisent pas cette syntaxe puisqu'en important un ensemble indéfini de noms, des noms déjà définis peuvent se retrouver masqués.

Notez qu'en général, importer `*` d'un module ou d'un paquet est déconseillé. Souvent, le code devient difficilement lisible. Son utilisation en mode interactif est acceptée pour gagner quelques secondes.

Si le nom du module est suivi par `as`, alors le nom suivant `as` est directement lié au module importé.

```
>>> import fibo as fib
>>> fib.fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

&gt;&gt;&gt;

Dans les faits, le module est importé de la même manière qu'avec `import fibo`, la seule différence est qu'il sera disponible sous le nom de `fib`.

C'est aussi valide en utilisant `from`, et a le même effet :

```
>>> from fibo import fib as fibonacci
>>> fibonacci(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

&gt;&gt;&gt;



En conséquence, si vous installez vos modules, vous devez recharger l'interpréteur. Si, au contraire, vous voulez tester interactivement, utilisez `importlib.reload()`, par exemple `import importlib ; importlib.reload(modulename)`.

### 6.1.1. Exécuter des modules comme des scripts

Lorsque vous exécutez un module Python avec :

```
python fibo.py <arguments>
```

le code du module est exécuté comme si vous l'aviez importé mais son `__name__` vaut `"__main__"`. Donc, en ajoutant ces lignes à la fin du module :

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

vous pouvez rendre le fichier utilisable comme script aussi bien que comme module importable, car le code qui analyse la ligne de commande n'est lancé que si le module est exécuté comme fichier « main » :

```
$ python fibo.py 50
0 1 1 2 3 5 8 13 21 34
```

Si le fichier est importé, le code n'est pas exécuté :

```
>>> import fibo
>>>
```

&gt;&gt;&gt;

C'est typiquement utilisé soit pour proposer une interface utilisateur pour un module, soit pour lancer les tests sur le module (exécuter le module en tant que script lance les tests).

### 6.1.2. Les dossiers de recherche de modules

Lorsqu'un module nommé par exemple `spam` est importé, il est d'abord recherché parmi les modules natifs. Les noms de ces modules sont listés dans `sys.builtin_module_names`. S'il n'est pas trouvé, l'interpréteur cherche un fichier nommé `spam.py` dans une liste de dossiers donnée par la variable `sys.path`. Par défaut, `sys.path` est initialisée à :

- le dossier contenant le script courant (ou le dossier courant si aucun script n'est donné) ;
- `PYTHONPATH` (une liste de dossiers, utilisant la même syntaxe que la variable shell `PATH`) ;
- La valeur par défaut, qui dépend de l'installation (incluant par convention un dossier `site-packages`, géré par le module `site`).

**Note:** sur les systèmes qui gèrent les liens symboliques, le dossier contenant le script courant est résolu après avoir suivi le lien symbolique du script. Autrement dit, le dossier contenant le lien symbolique n'est **pas** ajouté aux dossiers de recherche de modules.

Après leur initialisation, les programmes Python peuvent modifier leur `sys.path`. Le dossier contenant le script courant est placé au début de la liste des dossiers à rechercher, avant les dossiers de bibliothèques. Cela signifie qu'un module dans ce dossier, ayant le même nom qu'un module, sera chargé à sa place. C'est une erreur typique, à moins que ce ne soit voulu. Voir [Modules standards](#) pour plus d'informations.

### 6.1.3. Fichiers Python « compilés »



sion de Python) dans le dossier `__pycache__`. Par exemple, avec CPython 3.3, la version compilée de `spam.py` serait `__pycache__/spam.cpython-33.pyc`. Cette règle de nommage permet à des versions compilées par des versions différentes de Python de coexister.

Python compare les dates de modification du fichier source et de sa version compilée pour voir si le module doit être recompilé. Ce processus est entièrement automatique. Par ailleurs, les versions compilées sont indépendantes de la plateforme et peuvent donc être partagées entre des systèmes d'architectures différentes.

Il existe deux situations où Python ne vérifie pas le cache : le premier cas est lorsque le module est donné par la ligne de commande (cas où le module est toujours recompilé, sans même cacher sa version compilée) ; le second cas est lorsque le module n'a pas de source. Pour gérer un module sans source (où seule la version compilée est fournie), le module compilé doit se trouver dans le dossier source et sa source ne doit pas être présente.

Astuces pour les experts :

- vous pouvez utiliser les options `-O` ou `-OO` lors de l'appel à Python pour réduire la taille des modules compilés. L'option `-O` supprime les instructions `assert` et l'option `-OO` supprime aussi les documentations `__doc__`. Cependant, puisque certains programmes ont besoin de ces `__doc__`, vous ne devriez utiliser `-OO` que si vous savez ce que vous faites. Les modules « optimisés » sont marqués d'un `opt-` et sont généralement plus petits. Les versions futures de Python pourraient changer les effets de l'optimisation ;
- un programme ne s'exécute pas plus vite lorsqu'il est lu depuis un `.pyc`, il est juste chargé plus vite ;
- le module `compileall` peut créer des fichiers `.pyc` pour tous les modules d'un dossier ;
- vous trouvez plus de détails sur ce processus, ainsi qu'un organigramme des décisions, dans la [PEP 3147](#).

## 6.2. Modules standards

Python est accompagné d'une bibliothèque de modules standards, décrits dans la documentation de la Bibliothèque Python, plus loin. Certains modules sont intégrés dans l'interpréteur, ils proposent des outils qui ne font pas partie du langage mais qui font tout de même partie de l'interpréteur, soit pour le côté pratique, soit pour mettre à disposition des outils essentiels tels que l'accès aux appels système. La composition de ces modules est configurable à la compilation et dépend aussi de la plateforme cible. Par exemple, le module `winreg` n'est proposé que sur les systèmes Windows. Un module mérite une attention particulière, le module `sys`, qui est présent dans tous les interpréteurs Python. Les variables `sys.ps1` et `sys.ps2` définissent les chaînes d'invites principales et secondaires :

```
>>> import sys
>>> sys.ps1
'>>> '
>>> sys.ps2
'...'
>>> sys.ps1 = 'C> '
C> print('Yuck!')
Yuck!
C>
```

&gt;&gt;&gt;

Ces deux variables ne sont définies que si l'interpréteur est en mode interactif.

La variable `sys.path` est une liste de chaînes qui détermine les chemins de recherche de modules pour l'interpréteur. Elle est initialisée à un chemin par défaut pris de la variable d'environnement `PYTHONPATH` ou d'une valeur par défaut interne si `PYTHONPATH` n'est pas définie. `sys.path` est modifiable en utilisant les opérations habituelles des listes :

```
>>> import sys
```

&gt;&gt;&gt;



## 6.3. La fonction `dir()`

La fonction interne `dir()` est utilisée pour trouver quels noms sont définis par un module. Elle donne une liste de chaînes classées par ordre lexicographique :

```
>>> import fibo, sys
>>> dir(fibo)
['__name__', 'fib', 'fib2']
>>> dir(sys)
['__breakpointhook__', '__displayhook__', '__doc__', '__excepthook__',
'__interactivehook__', '__loader__', '__name__', '__package__', '__spec__',
'__stderr__', '__stdin__', '__stdout__', '__unraisablehook__',
'clear_type_cache', 'current_frames', 'debugmallocstats', 'framework',
'getframe', '_git', '_home', '_xoptions', 'abiflags', 'addaudithook',
'api_version', 'argv', 'audit', 'base_exec_prefix', 'base_prefix',
'breakpointhook', 'builtin_module_names', 'byteorder', 'call_tracing',
'callstats', 'copyright', 'displayhook', 'dont_write_bytecode', 'exc_info',
'excepthook', 'exec_prefix', 'executable', 'exit', 'flags', 'float_info',
'float_repr_style', 'get_asyncgen_hooks', 'get_coroutine_origin_tracking_depth',
'getallocatedblocks', 'getdefaultencoding', 'getdlopenflags',
'getfilesystemcodeerrors', 'getfilesystemencoding', 'getprofile',
'getrecursionlimit', 'getrefcount', 'getsizeof', 'getswitchinterval',
'gettrace', 'hash_info', 'hexversion', 'implementation', 'int_info',
'intern', 'is_finalizing', 'last_traceback', 'last_type', 'last_value',
'maxsize', 'maxunicode', 'meta_path', 'modules', 'path', 'path_hooks',
'path_importer_cache', 'platform', 'prefix', 'ps1', 'ps2', 'pycache_prefix',
'set_asyncgen_hooks', 'set_coroutine_origin_tracking_depth', 'setdlopenflags',
'setprofile', 'setrecursionlimit', 'setswitchinterval', 'settrace', 'stderr',
'stdin', 'stdout', 'thread_info', 'unraisablehook', 'version', 'version_info',
'warnoptions']
```

Sans paramètre, `dir()` liste les noms actuellement définis :

```
>>> a = [1, 2, 3, 4, 5]
>>> import fibo
>>> fib = fibo.fib
>>> dir()
['__builtins__', '__name__', 'a', 'fib', 'fibo', 'sys']
```

Notez qu'elle liste tous les types de noms : les variables, fonctions, modules, etc.

`dir()` ne liste ni les fonctions primitives, ni les variables internes. Si vous voulez les lister, elles sont définies dans le module `builtins` :

```
>>> import builtins
>>> dir(builtins)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException',
'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarning',
'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError',
'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationWarning',
'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False',
'FileExistsError', 'FileNotFoundError', 'FloatingPointError',
'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
'ImportWarning', 'IndentationError', 'IndexError', 'InterruptedError',
'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupError',
'MemoryError', 'NameError', 'None', 'NotADirectoryError', 'NotImplemented',
'NotImplementedError', 'OSError', 'OverflowError',
'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError',
'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning',
```



```

__builtin__ locals(), UNICODEDECODEERROR, UNICODEENCODING,
'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'UserWarning',
'ValueError', 'Warning', 'ZeroDivisionError', '_', '__build_class__',
'__debug__', '__doc__', '__import__', '__name__', '__package__', 'abs',
'all', 'any', 'ascii', 'bin', 'bool', 'bytearray', 'bytes', 'callable',
'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits',
'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec', 'exit',
'filter', 'float', 'format', 'frozenset', 'getattr', 'globals', 'hasattr',
'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'issubclass',
'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview',
'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property',
'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr', 'slice',
'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars',
'zip']

```

## 6.4. Les paquets

Les paquets sont un moyen de structurer les espaces de nommage des modules Python en utilisant une notation « pointée ». Par exemple, le nom de module `A.B` désigne le sous-module `B` du paquet `A`. De la même manière que l'utilisation des modules évite aux auteurs de différents modules d'avoir à se soucier des noms de variables globales des autres, l'utilisation des noms de modules avec des points évite aux auteurs de paquets contenant plusieurs modules tel que NumPy ou Pillow d'avoir à se soucier des noms des modules des autres.

Imaginez que vous voulez construire un ensemble de modules (un « paquet ») pour gérer uniformément les fichiers contenant du son et des données sonores. Il existe un grand nombre de formats de fichiers pour stocker du son (généralement identifiés par leur extension, par exemple `.wav`, `.aiff`, `.au`), vous avez donc besoin de créer et maintenir un nombre croissant de modules pour gérer la conversion entre tous ces formats. Vous voulez aussi pouvoir appliquer un certain nombre d'opérations sur ces sons : mixer, ajouter de l'écho, égaliser, ajouter un effet stéréo artificiel, etc. Donc, en plus des modules de conversion, vous allez écrire une myriade de modules permettant d'effectuer ces opérations. Voici une structure possible pour votre paquet (exprimée sous la forme d'une arborescence de fichiers) :

```

sound/                                Top-level package
  __init__.py                        Initialize the sound package
  formats/                          Subpackage for file format conversions
    __init__.py
    wavread.py
    wavwrite.py
    aiffread.py
    aiffwrite.py
    auread.py
    auwrite.py
    ...
  effects/                          Subpackage for sound effects
    __init__.py
    echo.py
    surround.py
    reverse.py
    ...
  filters/                          Subpackage for filters
    __init__.py
    equalizer.py
    vocoder.py
    karaoke.py
    ...

```

Lorsqu'il importe des paquets, Python cherche dans chaque dossier de `sys.path` un sous-dossier du nom du paquet.



later on the module search path. In the simplest case, `__init__.py` can just be an empty file, but it can also execute initialization code for the package or set the `__all__` variable, described later.

Les utilisateurs d'un module peuvent importer ses modules individuellement, par exemple :

```
import sound.effects.echo
```

charge le sous-module `sound.effects.echo`. Il doit alors être référencé par son nom complet.

```
sound.effects.echo.echofilter(input, output, delay=0.7, atten=4)
```

Une autre manière d'importer des sous-modules est :

```
from sound.effects import echo
```

charge aussi le sous-module `echo` et le rend disponible sans avoir à indiquer le préfixe du paquet. Il peut donc être utilisé comme ceci :

```
echo.echofilter(input, output, delay=0.7, atten=4)
```

Une autre méthode consiste à importer la fonction ou la variable désirée directement :

```
from sound.effects.echo import echofilter
```

Le sous-module `echo` est toujours chargé mais ici la fonction `echofilter()` est disponible directement :

```
echofilter(input, output, delay=0.7, atten=4)
```

Notez que lorsque vous utilisez `from package import element`, `element` peut aussi bien être un sous-module, un sous-paquet ou simplement un nom déclaré dans le paquet (une variable, une fonction ou une classe). L'instruction `import` cherche en premier si `element` est défini dans le paquet ; s'il ne l'est pas, elle cherche à charger un module et, si elle n'en trouve pas, une exception `ImportError` est levée.

Au contraire, en utilisant la syntaxe `import element.souselement.soussouselement`, chaque `element` sauf le dernier doit être un paquet. Le dernier `element` peut être un module ou un paquet, mais ne peut être ni une fonction, ni une classe, ni une variable définie dans l'élément précédent.

#### 6.4.1. Importer \* depuis un paquet

Qu'arrive-t-il lorsqu'un utilisateur écrit `from sound.effects import *` ? Idéalement, on pourrait espérer que Python aille chercher tous les sous-modules du paquet sur le système de fichiers et qu'ils seraient tous importés. Cela pourrait être long et importer certains sous-modules pourrait avoir des effets secondaires indésirables ou, du moins, désirés seulement lorsque le sous-module est importé explicitement.

La seule solution, pour l'auteur du paquet, est de fournir un index explicite du contenu du paquet. L'instruction `import` utilise la convention suivante : si le fichier `__init__.py` du paquet définit une liste nommée `__all__`, cette liste est utilisée comme liste des noms de modules devant être importés lorsque `from package import *` est utilisé. Il est de la responsabilité de l'auteur du paquet de maintenir cette liste à jour lorsque de nouvelles versions du paquet sont publiées. Un auteur de paquet peut aussi décider de ne pas autoriser d'importer `*` pour son paquet. Par exemple, le fichier `sound/effects/__init__.py` peut contenir le code suivant :

```
__all__ = ["echo", "surround", "reverse"]
```



Si `__all__` n'est pas définie, l'instruction `from sound.effects import *` n'importe *pas* tous les sous-modules du paquet `sound.effects` dans l'espace de nommage courant mais s'assure seulement que le paquet `sound.effects` a été importé (c.-à-d. que tout le code du fichier `__init__.py` a été exécuté) et importe ensuite les noms définis dans le paquet. Cela inclut tous les noms définis (et sous-modules chargés explicitement) par `__init__.py`. Sont aussi inclus tous les sous-modules du paquet ayant été chargés explicitement par une instruction `import`. Typiquement :

```
import sound.effects.echo
import sound.effects.surround
from sound.effects import *
```

Dans cet exemple, les modules `echo` et `surround` sont importés dans l'espace de nommage courant lorsque `from...import` est exécuté parce qu'ils sont définis dans le paquet `sound.effects` (cela fonctionne aussi lorsque `__all__` est définie).

Bien que certains modules ont été pensés pour n'exporter que les noms respectant une certaine structure lorsque `import *` est utilisé, `import *` reste considéré comme une mauvaise pratique dans du code à destination d'un environnement de production.

Rappelez-vous que rien ne vous empêche d'utiliser `from paquet import sous_module_specifique` ! C'est d'ailleurs la manière recommandée, à moins que le module qui fait les importations ait besoin de sous-modules ayant le même nom mais provenant de paquets différents.

#### 6.4.2. Références internes dans un paquet

Lorsque les paquets sont organisés en sous-paquets (comme le paquet `sound` par exemple), vous pouvez utiliser des importations absolues pour cibler des paquets voisins. Par exemple, si le module `sound.filters.vocoder` a besoin du module `echo` du paquet `sound.effects`, il peut utiliser `from sound.effects import echo`.

Il est aussi possible d'écrire des importations relatives de la forme `from module import name`. Ces importations relatives sont préfixées par des points pour indiquer leur origine (paquet courant ou parent). Depuis le module `surround`, par exemple vous pouvez écrire :

```
from . import echo
from .. import formats
from ..filters import equalizer
```

Notez que les importations relatives se fient au nom du module actuel. Puisque le nom du module principal est toujours `"__main__"`, les modules utilisés par le module principal d'une application ne peuvent être importés que par des importations absolues.

#### 6.4.3. Paquets dans plusieurs dossiers

Les paquets possèdent un attribut supplémentaire, `__path__`, qui est une liste initialisée avant l'exécution du fichier `__init__.py`, contenant le nom de son dossier dans le système de fichiers. Cette liste peut être modifiée, altérant ainsi les futures recherches de modules et sous-paquets contenus dans le paquet.

Bien que cette fonctionnalité ne soit que rarement utile, elle peut servir à élargir la liste des modules trouvés dans un paquet.

## Notes



